PROJECT HELIX

Key Requirements and Hands-On Validation Guide

GenomeBridge Clinical Systems Ltd | Version 1.0 | 4 August 2026

Core principle: Every phase follows one model: **Build - Execute - Investigate - Improve - Submit Evidence - Explain Decision.** No phase is a reading exercise. No phase is an essay. Every phase ends with something the candidate actually ran, built or produced using a real tool.

Workplace Setup Gate - Independent QE Workspace

BUILDS	A professional Quality Engineering workspace created entirely independently.
HANDS-ON TASKS	• Create the GitHub repository with branch strategy, PR template, issue templates and folder structure • Install and verify every tool: Node, Cypress, Playwright, k6, psql or Docker, Postman • Run one test in each framework to confirm the installation works • Configure GitHub Actions with a basic workflow that passes • Set up PostgreSQL - connect, create schema, run one verification query • Screenshot every tool version output as timestamped evidence
REAL WORLD	Every QE job starts with 'here is the stack, get yourself set up.' No one holds your hand. This is that moment, replicated exactly.
PROOF	Repository URL with real commit history. GitHub Actions run URL. Tool version screenshots. DB connection evidence. All submitted before Phase 1 begins.
REPO PATH	Repository root - github.com/alain-Sortnext/project-helix-quality-engineering

Phase 1 - Quality Discovery, Risk and Test Strategy

BUILDS	A risk register, system context map and a test strategy that decides what gets tested where and why.
HANDS-ON TASKS	• Log in to OpenMRS demo - actually explore patient search, registration and navigation; document what you find, not what the brief says you should find • Hit the HAPI FHIR R4 metadata endpoint in Postman; create one synthetic Patient resource and capture the generated ID • Build the system context map in Draw.io distinguishing fictional GenomeBridge from the real public systems • Build the risk register in Excel - 22 risks across 15 categories, each with likelihood x impact score, RAG status, proposed control and testing layer • Write the test-layer decision record: which risks go to Cypress, Playwright, API, SQL or stay manual • Create the requirements traceability matrix linking risks to requirements to planned tests

REAL WORLD	A QE joining a new team does exactly this in week one. The risk register is what hiring managers ask to see first. The decision record is what you defend in a technical interview.
PROOF	Draw.io file with real system boundaries. Risk register where risks reference what you actually observed. Traceability matrix. All committed to the Phase 1 branch.
REPO PATH	docs/quality-strategy/ | docs/architecture/ | docs/requirements/

Phase 2 - Exploratory Testing, Test Design and Defect Triage

BUILDS	Exploratory charters, a risk-based test scenario inventory (25 to 35 scenarios) and Jira defect records backed by evidence.
HANDS-ON TASKS	• Run 8 timed exploratory charters against OpenMRS: authentication, patient search, registration, duplicate risk, demographic editing, record navigation, keyboard-only, cross-browser • Apply formal test-design techniques to real inputs: date-of-birth boundaries (BVA), name field equivalence classes, session state transitions, role/action decision tables • Log defects in Jira with screenshot or console evidence - only what can be reproduced; at least one correctly classified as expected behaviour • Produce the risk-based test scenario inventory linking each scenario to a Phase 1 risk ID • Conduct triage for each material issue: severity, priority, reproducibility, release impact, recommended regression coverage
REAL WORLD	Exploratory testing before automation is standard QE practice. Formal test-design techniques separate a QE from a manual clicker. Defect triage is the daily conversation with developers.
PROOF	Jira defect IDs with timestamps. Charter notes showing what you tried and what happened. Screenshots showing actual browser state. Not invented.
REPO PATH	manual-testing/exploratory-charters/ | manual-testing/test-scenarios/ | manual-testing/defect-triage/

Phase 3 - FHIR API Quality Engineering

BUILDS	A Postman collection covering the full genomic referral workflow and Playwright TypeScript API tests automating the critical checks.
HANDS-ON TASKS	• Build the Postman collection: Patient, Practitioner, Organization, Consent, ServiceRequest, Specimen, Task, DiagnosticReport, AuditEvent, negative tests, cleanup • Chain the workflow: create Patient, capture ID, Consent, ServiceRequest, Specimen, Task - every resource linked using the generated ID from the prior step • Write pre-request scripts that generate unique synthetic identifiers so tests are repeatable after server resets • Translate high-value checks into Playwright TypeScript API tests with a reusable API client, schema assertions, captured IDs and cleanup • Analyse a fictional API compatibility diff - identify removed fields, renamed fields, type changes and consumer impact

REAL WORLD	API testing is the most transferable QE skill in the UK market. Postman collections get opened in interviews. Playwright request contexts are exactly what modern SDETs build.
PROOF	Postman collection export with real HAPI FHIR resource IDs in response examples. Playwright test output showing TypeScript compilation and actual HTTP responses.
REPO PATH	api-testing/postman/ | api-testing/playwright-api/ | api-testing/schemas/

Phase 4 - Health Data Quality, SQL and Test-Data Management

BUILDS	A local PostgreSQL database with synthetic healthcare data imported, a SQL validation suite, reusable data utilities and a test-data policy.
HANDS-ON TASKS	• Create the database independently: local PostgreSQL, Docker, Neon or Supabase • Import a Synthea CSV dataset subset and the FHIR resources created in Phase 3 • Write SQL validation queries: duplicate patient IDs, missing names, future birth dates, orphan service requests, specimens without patients, broken foreign keys • Build TypeScript utilities for unique patient generation, test-run tagging and cleanup • Produce a data-quality report: rule, query, row-count result, severity, business implication and recommended action • Write the synthetic test-data policy covering naming convention, tagging, retention, prohibited data and environment separation
REAL WORLD	Healthcare data quality is a specific, valued skill in UK HealthTech QA. Test-data management policies are what mature QE teams have and junior ones lack.
PROOF	SQL query output with actual row counts from real imported data. Database connection evidence. Data-quality report referencing specific figures from the dataset.
REPO PATH	database/schema/ | database/validation/ | database/quality-rules/ | database/reports/

Phase 5 - Cypress Regression Engineering

BUILDS	A focused, reliable Cypress suite with network interception, reliability measurement and a framework allocation decision.
HANDS-ON TASKS	• Build reusable custom commands: loginAsAdmin, searchPatient, generateSyntheticPatient, assertValidationMessage, captureNetworkFailure • Write 15 or more tests across authentication, patient search, validation, navigation, session and error handling • Use cy.intercept() to wait on real requests, assert response status and inspect responses • Run the suite repeatedly - calculate first-run pass rate, final pass rate, intermittent failures and failures by root cause • Fix instability: hardcoded waits, fragile selectors, shared test state, fixed patient IDs • Produce the framework allocation matrix: which tests stay in Cypress, which move to Playwright, which go to API, which are removed

REAL WORLD	Almost every UK QE job with Cypress involves inheriting a flaky suite and being asked to improve it. The reliability measurement is what you present to engineering leadership.
PROOF	Cypress run output with timestamps and pass/fail counts. Before and after reliability figures. The allocation matrix with rationale per test.
REPO PATH	cypress/e2e/ | cypress/support/ | cypress/fixtures/ | reports/phase-5/

Phase 6 - Playwright Cross-Browser Quality Engineering

BUILDS	A modern Playwright suite covering five critical journeys, accessibility checks with axe-core and cross-browser results across Chromium, Firefox and WebKit.
HANDS-ON TASKS	• Journey 1: login, patient search with dynamic criteria, open record, verify identity markers • Journey 2: registration form with boundary and invalid values, verify validation, capture identifier dynamically • Journey 3: same record-access flow in Chromium, Firefox and WebKit; record browser-specific differences • Journey 4: logout, return navigation, session recovery • Journey 5: create synthetic data via FHIR API in the same test run, use the generated ID in a browser test, verify correct record • Add axe-core accessibility checks to login, search, registration and error states • Compare Cypress and Playwright on setup, debugging, selectors, browser coverage, network handling, API support and reporting
REAL WORLD	Playwright is the fastest-growing automation tool in UK QE hiring (IT Jobs Watch: 201 UK vacancies, median GBP 60,000). Journey 5 (API-assisted setup) is the specific pattern modern SDETs use.
PROOF	Playwright HTML report with cross-browser results. axe-core violation counts per page. Journey 5 requires a real FHIR resource ID created in the same test run.
REPO PATH	playwright/tests/ | playwright/fixtures/ | playwright/pages/ | accessibility/

Phase 7 - CI/CD, Performance and Quality Signals

BUILDS	A progressive GitHub Actions pipeline, k6 performance tests against a local mock, failure-injection tests and a quality scorecard spreadsheet.

HANDS-ON TASKS	• Wire four workflows: PR checks (typecheck + lint + smoke), main-branch regression, scheduled cross-browser and accessibility, manual release trigger • Run the pipeline, capture GitHub Actions run URLs as evidence and fix failures before submitting • Create a local mock endpoint using Mockoon or JSON Server representing referral submission; do not load-test the public systems • Write k6 scripts: baseline, expected load, short stress spike, recovery; capture p50/p95/p99 latency and error rate • Inject failures via Playwright interception or the local mock: 400, 401, 404, 500, slow response, malformed response; verify graceful handling • Define and calculate quality metrics: pass rate, flaky-test rate, defect density, risk coverage, automated coverage of prioritised scenarios
REAL WORLD	A QE who can set up GitHub Actions and explain the pipeline decisions is immediately more valuable. k6 appears explicitly in UK job ads (Collinson, Genomics England). Quality metrics go to the release review.
PROOF	GitHub Actions run URLs. k6 terminal output with real latency figures. Failure-injection test output. Quality scorecard spreadsheet built from actual results.
REPO PATH	.github/workflows/ | performance/k6/ | metrics/ | reports/phase-7/

Phase 8 - Release Readiness and Quality Leadership

BUILDS	A final Go / Conditional Go / No-Go recommendation backed by all evidence from Phases 1 to 7, with written responses to six stakeholder challenges.
HANDS-ON TASKS	• Reconcile all evidence from every previous phase - classify every finding as confirmed defect, automation defect, data-quality problem, environment instability or accepted limitation • Update the risk register: every material risk gets current status, evidence reference, residual risk and release impact • Produce the final quality scorecard covering 12 dimensions: critical workflow status, open defects, API, data quality, Cypress stability, Playwright stability, cross-browser, accessibility, performance, CI, environmental constraints, traceability • Write the Go/No-Go recommendation - Conditional Go requires specific conditions, named owners, deadlines, monitoring triggers and rollback criteria • Write responses to six named stakeholder challenges from Daniel Brooks, Maya Patel, Laura Okafor and James Wright • Produce the 30-day quality improvement plan covering automation, accessibility, environment, data and metrics • Tag the final GitHub release
REAL WORLD	The release review is the moment QEs are most visible to leadership. A Conditional Go with specific conditions and owners is a senior-level skill.
PROOF	Recommendation must reference specific figures from Phases 1 to 7. Every condition in a Conditional Go names a Jira ticket and an owner. Risk register delta from Phase 1 to Phase 8 is visible.

REPO PATH	release/ | metrics/scorecards/ | reports/phase-8/

Cross-Phase Connections

From	Carries Into	What It Carries
Phase 1 risk IDs	Phase 2 charter missions	Every charter targets a named risk from the register
Phase 2 defect IDs	Phase 3 API test objectives	Browser defects drive FHIR API investigation
Phase 3 FHIR resource IDs	Phase 4 SQL validation	Created resources appear in the database import
Phase 4 naming conventions	Phase 5 Cypress test data	Unique-run identifiers prevent shared-state failures
Phase 5 allocation matrix	Phase 6 Playwright scope	Playwright only covers what the matrix says to move
Phase 6 Playwright results	Phase 7 CI/CD triggers	Cross-browser suite feeds the scheduled workflow
Phases 1 to 7 - everything	Phase 8 scorecard	Nothing in Phase 8 is invented; it is all reconciled

What This Portfolio Proves to a UK HealthTech Employer

Artefact	What a Hiring Manager Reads From It
Risk register and test strategy	This person can join a team and immediately structure work around real risks, not test c
FHIR Postman collection	This person understands healthcare interoperability well enough to test it, not just descri
SQL data-quality report	This person can validate a database, not just UI flows
Cypress reliability analysis and allocation matrix	This person inherited a messy suite and improved it, and can explain why
Playwright cross-browser and Journey 5	This person uses API-assisted setup, which means they know what makes automation b
GitHub Actions pipeline	This person can set up CI/CD without being handed a template
Go/No-Go recommendation	This person can be in the release review room and hold their own

Indicative UK salary range based on the referenced market research: approximately GBP 45,000 to GBP 70,000 for relevant mid-level QA Automation Engineer and SDET roles in UK HealthTech. Source: IT Jobs Watch aggregate data, Reed.co.uk, LinkedIn UK observed August 2026. Figures are indicative and should be verified against current market data before use.